# Requirements Verification Artifacts

Requirement IDs 1–39 | Proposed draft for team/GTA review

**SCOPE STATUS** The workbook leaves Sprint No. blank for all 39 rows. This document covers the full requirements backlog;

### Purpose

Turn each source requirement into a reviewable verification package: expected behavior, acceptance criteria, example inputs and expected outputs, constraints, dependencies, a proposed definition of done, and a place to attach real evidence. The example tests below are proposals only; none were executed, and no implementation results are claimed.

### How to review this artifacts

- Each requirement section preserves the source row wording and adds a proposed, testable interpretation.
- Items that need a policy choice (such as currency rates, variance convention, precision, or supported equation class) call it out in the validation/open-decisions section.
- After the team agrees sprint scope, enter the approved sprint number in the google sheet and copy only those approved requirements into the Sprint 1 submission package.

**Google Sheet:** Project 3: Requirements Open source workbook

## Requirement register

| ID | Requirement | Points | Priority | Sprint No. |
|---|---|---|---|---|
| 01 | Expression parser | 1 | 1 | 1 |
| 02 | Arbitrary-precision calculation engine | 1 | 2 | 1 |
| 03 | Persistent session history and Ans recall | 1 | 3 | 1 |
| 04 | Unit conversion | 2 | 4 | 1 |
| 05 | Clean, user-friendly UI layout | 2 | 5 | 1 |
| 06 | 2D single-variable graphing | 2 | 6 | 1 |
| 07 | Calculation mode switcher | 1 | 7 | 1 |
| 08 | Function search and category filter | 1 | 8 | 1 |
| 09 | Financial calculations | 2 | 9 | 2 |
| 10 | Calculation-step and formula breakdown | 2 | 10 | 2 |
| 11 | Helpful input checker | 1 | 11 | 2 |
| 12 | Undo and redo for expression editing | 1 | 12 | 2 |
| 13 | Number theory tools | 3 | 13 | 2 |
| 14 | Displayed precision controls | 2 | 14 | 2 |
| 15 | Tolerance calculator | 2 | 15 | 2 |
| 16 | Statistical calculations | 3 | 16 | 2 |
| 17 | Fraction calculator | 3 | 17 | 2 |

| ID | Requirement | Points | Priority | Sprint No. |
|---|---|---|---|---|
| 18 | Percentage calculator | 2 | 18 | 2 |
| 19 | Graph customization | 3 | 19 | 2 |
| 20 | Basic algebra equation solver | 5 | 20 | 2 |
| 21 | Favorite calculations | 2 | 21 | 2 |
| 22 | Display customization | 2 | 22 | 2 |
| 23 | Export calculation history | 3 | 23 | 2 |
| 24 | Graph comparison | 3 | 24 | 2 |
| 25 | Financial scenario comparison | 3 | 25 | 2 |
| 26 | Programmer mode | 2 | 26 | 2 |
| 27 | Matrix operations | 2 | 27 | 2 |
| 28 | Calculus tools | 1 | 28 | 2 |
| 29 | Complex-number mode | 1 | 29 | 2 |
| 30 | Find graph intercepts | 2 | 30 | Unassigned |
| 31 | Constants library | 2 | 31 | Unassigned |
| 32 | Save graphs as PNG | 1 | 32 | Unassigned |
| 33 | Notes on saved calculations | 1 | 33 | Unassigned |
| 34 | Light and dark mode | 1 | 34 | Unassigned |
| 35 | Region-dependent number formatting | 1 | 35 | Unassigned |
| 36 | Right-click shortcut menu | 1 | 36 | Unassigned |
| 37 | Thousands separator toggle | 1 | 37 | Unassigned |
| 38 | Degree/radian angle toggle | 1 | 38 | Unassigned |
| 39 | Scientific notation toggle | 1 | 39 | Unassigned |

# Requirement 01 — Expression parser

Proposed / hypothetical • Story points: 1 • Priority: 1 • Sprint: 1

**Source description:** A core expression parser handling basic arithmetic, scientific functions, and parenthetical evaluation.

## What it should do

Read a calculator expression, apply the agreed grammar and operator precedence, and pass a structured expression to the calculation engine. Support arithmetic, parentheses, and the scientific functions the team selects.

## Acceptance criteria

- Multiplication and division take precedence over addition and subtraction; parentheses override precedence.
- Recognized scientific functions are parsed with the agreed argument and angle-unit conventions.
- Equivalent valid expressions produce the same expression structure regardless of harmless whitespace.

## Example test cases — proposed, not run

| Test ID | Input / action | Expected output |
|---|---|---|
| R01-T1 | 2 + 3 × 4 | 14; multiplication is evaluated before addition. |
| R01-T2 | (2 + 3) × 4 | 20; the parenthesized sum is evaluated first. |
| R01-T3 | sin(π/2) | 1, if radians and the listed sin function are selected; confirm angle units with the team. |

**UI / workflow notes:** One expression entry area; preserve the entered expression while showing the parsed/evaluated result or a linked diagnostic.

**Validation, limitations, and open decisions:** Reject unsupported tokens and malformed syntax with a location-aware message. Function list, implicit multiplication, exponent syntax, angle units, and numeric limits are not defined in the source and need agreement.

**Dependencies and risks:** Feeds the calculation engine (ID 2), input diagnostics (ID 11), step breakdown (ID 10), algebra (ID 20), and graph/calculus inputs (IDs 6, 28, 30). Risk: parser and engine conventions can drift.

**Proposed definition of done:** Team-approved grammar is documented; listed valid/invalid expression tests pass; precedence and function behavior are reproducible; diagnostics and evidence are attached.

## Requirement 01 – Test Evidence

**Implementation**: Expression parser supporting arithmetic operations, operator precedence, parentheses, scientific functions, and syntax error detection.

**Test Date**: November 8, 2026 (by Ivan Kullaya)

**Test Environment**: Web browser (Google Chrome)

| Test ID | Test Performed | Expected result | Status |
|---|---|---|---|
| R01-T1 | 2 + 3 x 4 | 14 | Pass |
| R01-T2 | (2 + 3) x 4 | 20 | Pass |
| R01-T3 | sin(π/2) | 1 | Pass |
| R01-T4 | 2 + (3 x 4) | 14 | Pass |
| R01-T5 | 2 + 3 x | Error identifying incomplete expression | Pass |
| R01-T6 | 2 @ 3 | Error identifying unsupported character | Pass |
| R01-T7 | 2+3×4 and 2 + 3 × 4 | Both return 14 | Pass |

**Evidence:** GitHub R01 Evidence Folder

**Implementation Details:** The parser uses tokenization and recursive parsing to interpret mathematical expressions. It supports addition, subtraction, multiplication, division, exponentiation, parentheses, and scientific functions such as sin, cos, tan, sqrt, ln, and log. It also supports implicit multiplication and recognizes invalid expressions with error messages identifying the location of the issue.

**Result:** All four tests passed, confirming that Requirement 01 meets its acceptance criteria.

# Requirement 02 — Arbitrary-precision calculation engine

Proposed / hypothetical • Story points: 1 • Priority: 2 • Sprint: 1

**Source description:** A basic arbitrary-precision calculation engine ensuring accurate decimal results without standard floating-point rounding errors.

## What it should do

Evaluate parsed numeric operations with a representation and precision policy that avoid ordinary binary floating-point surprises for decimal inputs.

## Acceptance criteria

- Decimal operations preserve the intended decimal result under the approved precision policy.
- Displayed rounding does not silently change the engine's stored value.
- Overflow, division by zero, and precision-limit behavior are defined and reported safely.

## Example test cases — proposed, not run

| Test ID | Input / action | Expected output |
|---|---|---|
| R02-T1 | 0.1 + 0.2 | 0.3 exactly under the proposed decimal semantics. |
| R02-T2 | 1 ÷ 3, then multiply the stored result by 3 | 1 within the documented internal precision policy; the display may round separately. |

**UI / workflow notes:** Show the evaluated result in the shared result area; if precision limits apply, make them discoverable without replacing the input.

**Validation, limitations, and open decisions:** The source does not specify precision, rounding mode, exponent range, or whether arbitrary precision is exact rational, decimal, or configurable. Agree these limits before final acceptance.

**Dependencies and risks:** Consumes the parser output (ID 1) and supports most numeric features. Risk: using display-rounded values in later calculations would violate the preservation goal in ID 14.

**Proposed definition of done:** The numeric representation and limits are documented; decimal and precision-boundary cases pass; downstream features use the full stored result; evidence records actual outputs.

## Requirement 02 – Test Evidence

**Implementation**: Arbitrary-precision calculation engine intended to support accurate decimal arithmetic, internal precision preservation, and safe handling of mathematical errors.

**Test Date**: November 9, 2026 (by Ivan Kullaya)

**Test Environment**: Web browser (Google Chrome)

| Test ID | Test Performed | Expected result | Status |
|---|---|---|---|
| R02-T1 | Calculate 0.1 + 0.2 | Result is exactly 0.3 under the approved decimal precision policy | Pass |
| R02-T2 | Calculate 1 ÷ 3, then multiply the stored result by 3 | Result is 1 within the documented internal precision policy | Fail / Needs Review |
| R02-T3 | Calculate 10 ÷ 0 | Division-by-zero error message appears without crashing the calculator | Pass |
| R02-T4 | Enter a calculation exceeding the supported numeric range | Calculator safely reports an overflow or precision-limit error | Pass |
| R02-T5 | Calculate 0.1 + 0.2 + 0.3 | Result is 0.6 under the approved decimal precision policy | Pass |

**Evidence:** GitHub R02 Evidence Folder

**Implementation Details:**

The calculation engine is intended to evaluate mathematical expressions using a numeric representation that minimizes or eliminates standard floating-point rounding errors.

The implementation must define its internal numeric representation, supported precision, rounding rules, and numeric limits. Display formatting should remain separate from internal calculations so that subsequent operations use the stored result rather than a rounded display value.

Division by zero, overflow, and unsupported precision cases should produce appropriate error messages without interrupting the calculator's functionality.

**Result:** Requirement 02 is partially verified. Additional testing and potential improvements to internal precision handling are needed before the requirement can be marked complete.

# Requirement 03 — Persistent session history and Ans recall

Proposed / hypothetical • Story points: 1 • Priority: 3 • Sprint: 1

**Source description:** A simple persistent session history allowing users to recall past inputs and outputs via a standard "Ans" key.

### What it should do

Keep a chronological list of calculations and results that survives the agreed persistence boundary, and let the next expression refer to the previous result through Ans.

### Acceptance criteria

- A completed calculation appears with its original input and result in history.
- Ans resolves to the prior result, not the formatted history label.
- History remains available after the persistence event selected by the team, such as app restart.

### Example test cases — proposed, not run

| Test ID | Input / action | Expected output | Status |
|---|---|---|---|
| R03-T1 | Evaluate 2 + 3, then Ans + 4 | History contains 2 + 3 → 5; second result is 9. | Not run |
| R03-T2 | Evaluate 7 × 6, close and reopen the app | History contains the saved 7 × 6 → 42 if restart persistence is approved. | Not run |

**UI / workflow notes:** History list shows expression and result; selecting an item may recall its expression. Keep Ans available in the expression workflow.

**Validation, limitations, and open decisions:** Define what counts as a session, history retention/clear behavior, ordering, and whether Ans resets after errors or mode changes. Avoid saving incomplete expressions as completed results.

**Dependencies and risks:** Uses calculation results from ID 2 and persistence storage. Mode preservation (ID 7), favorites (ID 21), export (ID 23), and notes (ID 33) may reuse history records.

**Proposed definition of done:** History and Ans rules are agreed; recall and persistence cases pass; ordering and error behavior are documented; evidence shows saved records and actual results.

## Requirement 03 – Test Evidence

**Implementation**: Persistent calculation history and Ans recall using JavaScript and localStorage.

**Test Date**: November 8, 2026 (by Ivan Kullaya)

**Test Environment**: Web browser (Google Chrome)

| Test ID | Test Performed | Expected result | Status |
|---|---|---|---|
| R03-T1 | Calculate 2 + 3, then Ans + 4 | Results are 5 and 9; calculations appear in history | Pass |
| R03-T2 | Calculate 7 × 6, then close and reopen the calculator | History still contains 7 × 6 = 42 | Pass |
| R03-T3 | Select a previous calculation from history | Previous expression is restored to the input field | Pass |
| R03-T4 | Clear calculation history | Saved history is removed | Pass |

**Evidence:** GitHub R03 Evidence Folder

**Implementation Details:** Calculation history is stored using browser localStorage. Each successful calculation saves the original expression and numerical result. The Ans variable stores the previous successful result for use in future calculations. History is limited to the 100 most recent calculations.

**Result:** All four test passed, confirming that Requirement 03 meets its acceptance criteria.

## Requirement 04 — Unit conversion

**Proposed / hypothetical • Story points: 2 • Priority: 4 • Sprint: 1**

**Source description:** A standard conversion module handling the most common length, weight, volume, temperature, and basic currency units.

### What it should do

Convert a value between team-approved common units in length, mass/weight, volume, temperature, and basic currencies, showing the source and converted values with units.

### Acceptance criteria

- Conversions use the correct category and unit-specific formula.
- Temperature conversions apply offsets as well as scale factors.
- Currency conversions identify the rate and its date/source when rates are time-dependent.

### Example test cases — proposed, not run

| Test ID | Input / action | Expected output |
|---|---|---|
| R04-T1 | 1 m → cm | 100 cm. |
| R04-T2 | 0 °C → °F | 32 °F. |
| R04-T3 | $10 USD → EUR | Converted amount = 10 × the selected USD→EUR rate; show rate/date/source. Numeric result depends on the team-approved rate source and timestamp. |

**UI / workflow notes:** Provide value, source-unit, destination-unit, and converted-result fields; group units by category and make the active category clear.

**Validation, limitations, and open decisions:** Reject incompatible units and non-finite values. The supported unit list, currency-rate provider, offline behavior, rate timestamp, and rounding are not specified; agree them before final acceptance.

**Dependencies and risks:** Needs unit definitions and a conversion engine; currency needs an agreed external or stored rate source. Risk: stale currency rates can make a plausible but misleading result.

**Proposed definition of done:** Approved unit lists/formulas and currency policy are documented; length, temperature, invalid-category, and rate-attribution cases pass; evidence includes actual rate metadata.

## Requirement 04 – Test Evidence

**Implementation**: Unit conversion module supporting length, weight, volume, temperature, and currency conversions with selectable source and destination units.

**Test Date**: November 8, 2026 (by Ivan Kullaya)

**Test Environment**: Web browser (Google Chrome)

| Test ID | Test Performed | Expected result | Status |
|---|---|---|---|
| R04-T1 | Convert 1 meter to centimeters | 100 cm | Pass |
| R04-T2 | Convert 0°C to Fahrenheit | 32°F | Pass |
| R04-T3 | Convert 10 USD to EUR | Correct converted amount using the selected exchange rate, with rate, date, and source displayed | Pass |
| R04-T4 | Convert 5 kilograms to grams | 5,000 g | Pass |
| R04-T5 | Convert 2 liters to milliliters | 2,000 mL | Pass |
| R04-T6 | Attempt conversion between incompatible units | Error message or prevention of invalid conversion | Pass |

**Evidence:** GitHub R04 Evidence Folder

**Implementation Details:**

The unit conversion module is intended to provide conversions across five categories: length, weight, volume, temperature, and currency. Users select a conversion category, enter a numerical value, choose the source and destination units, and receive the converted result.

Length, weight, and volume conversions use predefined conversion factors. Temperature conversions use formulas that account for both scale and offset. Currency conversions require an approved exchange-rate source and should display the rate and its associated date.

The module should also prevent incompatible conversions and reject invalid numerical inputs.

**Result:** All four tests passed, confirming that Requirement 04 meets its acceptance criteria.

# Requirement 05 — Clean, user-friendly UI layout

Proposed / hypothetical • Story points: 2 • Priority: 5 • Sprint: 1

**Source description:** A clean, user-friendly UI layout.

### What it should do

Present the calculator's primary input, result, key actions, and available modes in a readable layout that users can operate without guessing where results or controls are.

### Acceptance criteria

- Input, result, and primary actions have clear labels and visible focus/selection.
- A new user can complete the agreed basic calculation task without an instruction from the tester.
- Layout remains usable at the team-approved window sizes and text scaling.

### Example test cases — proposed, not run

| Test ID | Input / action | Expected output |
|---|---|---|
| R05-T1 | Ask a first-time tester to enter 2 + 3 and find the result | Tester completes the task and identifies 5 without coaching; record task steps and issues. |
| R05-T2 | Resize to the smallest supported window and increase text size | Primary input, result, and actions remain visible or reachable without overlap. |

**UI / workflow notes:** This requirement is the overall layout foundation. Record target platforms/window sizes and keyboard-accessible paths in the review notes.

**Validation, limitations, and open decisions:** "Clean" and "user-friendly" are subjective until usability targets, accessibility expectations, and supported sizes are agreed.

**Dependencies and risks:** Provides the shared shell for all modes and features (IDs 7, 8, 22, 34). Risk: feature-specific controls may overcrowd the primary workflow.

**Proposed definition of done:** Team-approved usability checks pass on target sizes; labels, focus, and navigation are verified; tester observations and screenshots are attached.

## Requirement 05 – Test Evidence

**Implementation**: Persistent calculation history and Ans recall using JavaScript and localStorage.

**Test Date**: November 8, 2026 (by Ivan Kullaya)

**Test Environment**: Web browser (Google Chrome)

| Test ID | Test Performed | Expected result | Status |
|---|---|---|---|
| R05-T1 | First-time user enters 2 + 3 and presses = | User completes calculation and identifies result 5 without assistance | Pass |
| R05-T2 | Resize browser to smallest supported window | Calculator buttons, input, and result remain accessible without overlap | Pass |
| R05-T3 | Resize browser to smallest supported window | Interactive controls have visible focus indicators | Pass |
| R05-T4 | Increase browser text scaling to 200% | Calculator remains readable and functional without overlapping controls | Pass |

**Evidence:** GitHub R05 Evidence Folder

**Implementation Details:** The calculator interface is built using HTML and CSS with a centered layout, organized button grids, and separate input and result displays. Scientific functions are grouped separately from basic arithmetic controls. CSS Grid maintains consistent button spacing, while responsive sizing allows the calculator to adapt to different browser widths. The interface also includes visible keyboard focus indicators, light and dark color schemes, and accessible labels for important controls.

**Result:** All four tests passed, confirming that Requirement 05 meets its acceptance criteria.

# Requirement 06 — 2D single-variable graphing

Proposed / hypothetical • Story points: 2 • Priority: 6 • Sprint: 1

**Source description:** A 2D graphing module, able to plot single variable equations on a canvas viewer with controls such as pan and zoom.

### What it should do

Plot a selected single-variable function on a two-dimensional coordinate plane and let users move and scale the visible graph.

### Acceptance criteria

- A valid function is plotted over a visible x-range with correctly placed axes.
- Pan changes the displayed coordinate window without changing the function.
- Zoom changes the displayed scale and retains the plotted function.

### Example test cases — proposed, not run

| Test ID | Input / action | Expected output |
|---|---|---|
| R06-T1 | Plot y = 2x over a range containing 0 and 1 | Line passes through (0, 0) and (1, 2), within the agreed rendering tolerance. |
| R06-T2 | Pan right, then zoom in once | Viewport center/range changes as indicated by the controls; plotted line remains y = 2x. |

**UI / workflow notes:** Show graph canvas, coordinate axes/ticks, function identity, and discoverable pan/zoom controls. State the active range.

**Validation, limitations, and open decisions:** Define supported functions, sampling/tolerance, discontinuity handling, default ranges, and pan/zoom increments. A graph is a visual approximation, not proof of symbolic correctness.

**Dependencies and risks:** Uses expression syntax from ID 1 and a graph renderer; customization (ID 19), comparison (ID 24), intercepts (ID 30), and PNG export (ID 32) build on this view.

**Proposed definition of done:** Approved function set and plotting tolerance are documented; known-point and viewport tests pass; screenshots show axes, correct plot, and pan/zoom state.

## Requirement 06 – Test Evidence

**Implementation**: Persistent calculation history and Ans recall using JavaScript and localStorage.

**Test Date**: November 8, 2026 (by Ivan Kullaya)

**Test Environment**: Web browser (Google Chrome)

| Test ID | Test Performed | Expected result | Status |
|---|---|---|---|
| R06-T1 | Plot y = 2x over a range containing 0 and 1 | Graph displays a straight line passing through (0, 0) and (1, 2) | Blocked — Unable to Test |
| R06-T2 | Pan the graph to the right | Viewport shifts horizontally while preserving the plotted function | Blocked — Unable to Test |
| R06-T3 | Zoom in on the graph | Graph scale changes while maintaining the correct function | Blocked — Unable to Test |
| R06-T4 | Plot $y = x^2$ | Graph displays a parabola with its vertex at (0, 0) | Blocked — Unable to Test |
| R06-T5 | Zoom out and return to the original viewing range | Graph remains visible with correctly positioned axes and coordinates | Blocked — Unable to Test |

**Evidence:** GitHub R06 Evidence Folder

**Implementation Details:**

The 2D graphing module is intended to allow users to enter single-variable equations and display their corresponding graphs on a coordinate plane. The module should support interactive controls for panning and zooming while maintaining the mathematical accuracy of the plotted function.

The graphing functionality is currently experiencing implementation issues that prevent the planned tests from being completed. As a result, the graphing feature has not yet been verified against its acceptance criteria.

The test cases have been documented so they can be executed once the functionality is restored.

**Result:** Requirement 06 is not yet verified. Testing is currently blocked because the graphing module requires additional fixes. All planned test cases have been documented but remain unexecuted.

Once the graphing functionality is restored, the assigned developer can complete the tests, record the actual results, and attach screenshots demonstrating successful plotting, panning, and zooming.

# Requirement 07 — Calculation mode switcher

**Proposed / hypothetical • Story points: 1 • Priority: 7 • Sprint: 1**

**Source description:** A calculation mode switcher that allows users to choose between basic, scientific, financial, and conversion modes. Current input/session history should be preserved.

## What it should do

Switch among basic, scientific, financial, and conversion modes while retaining the current draft input and the existing session history.

## Acceptance criteria

- Each named mode can be selected and is visibly identified.
- Switching modes does not erase or silently alter the current input.
- Existing session history remains accessible after a mode change.

## Example test cases — proposed, not run

| Test ID | Input / action | Expected output |
|---|---|---|
| R07-T1 | Enter draft 12 × (3 + 1), switch Basic → Scientific → Basic | Draft remains exactly 12 × (3 + 1), unless team-approved mode validation reports it unsupported. |
| R07-T2 | Create history item 2 + 3 → 5, switch to Conversion | History item remains available and unchanged. |

**UI / workflow notes:** Use a persistent mode selector and visibly indicate the active mode; distinguish a draft from a completed history entry.

**Validation, limitations, and open decisions:** Define behavior for expressions unsupported in the destination mode, and whether mode-specific settings persist independently.

**Dependencies and risks:** Requires the shared UI (ID 5), mode features (IDs 1, 4, 9), and session history (ID 3). Risk: mode switch may re-interpret rather than preserve input.

**Proposed definition of done:** Mode names and transition behavior are agreed; draft and history preservation tests pass; screenshots demonstrate each mode and state retention.

## Requirement 07 – Test Evidence

**Implementation**: Calculation mode switcher allowing users to navigate between Basic, Scientific, Financial, and Conversion modes while preserving current input and calculation history.

**Test Date**: November 9, 2026 (by Ivan Kullaya)

**Test Environment**: Web browser (Google Chrome)

| Test ID | Test Performed | Expected result | Status |
|---|---|---|---|
| R07-T1 | Enter 12 × 3 + 1, switch Basic → Scientific → Basic | Original expression remains unchanged | Pass |
| R07-T2 | Calculate 2 + 3 = 5, then switch to Conversion mode | Previous calculation remains available in history | Pass |
| R07-T3 | Switch between Basic, Scientific, Financial, and Conversion modes | Each mode loads correctly and is visibly identified | Fail |
| R07-T4 | Enter an unfinished expression, then switch modes | Draft expression is preserved without being cleared or modified | Pass |
| R07-T5 | Switch modes multiple times and return to the original mode | Current input and saved calculation history remain accessible | Partial Pass |

**Evidence:** GitHub R07 Evidence Folder

**Implementation Details:**

The calculation mode switcher is intended to provide navigation between Basic, Scientific, Financial, and Conversion modes using a shared calculator interface.

Each mode should be clearly identified through the mode selector or active mode indicator. Switching modes should preserve the current expression without silently changing its contents.

Calculation history should remain available across mode changes, allowing users to access previously completed calculations regardless of the active mode.

The implementation should also define how unsupported expressions are handled when switching to a mode that does not support certain functions.

**Result:** Requirement 02 is partially verified. The mode switcher successfully preserves user input and calculation history. However, improvements are needed to ensure each calculation mode displays its corresponding interface.

The Financial and Conversion modes should display their respective tools when selected, rather than requiring users to access them through secondary buttons in Scientific mode. Additionally, secondary panels should close or hide automatically when switching to a different mode.

These issues should be addressed and retested before Requirement 07 is considered fully complete.

# Requirement 08 — Function search and category filter

Proposed / hypothetical • Story points: 1 • Priority: 8 • Sprint: 1

**Source description:** A category filter system that allows the user to search for functions by category.

## What it should do

Let users find calculator functions by text and by a category label such as the team-approved algebra, statistics, finance, or trigonometry groups.

## Acceptance criteria

- Choosing a category limits results to functions tagged with that category.
- Search narrows the visible functions using an agreed name/description matching rule.
- No-match results are clearly communicated and do not appear as valid functions.

## Example test cases — proposed, not run

| Test ID | Input / action | Expected output |
|---|---|---|
| R08-T1 | Select Trigonometry and search “sin” | sin appears if it is in the approved function catalog; unrelated functions are excluded. |
| R08-T2 | Search for “no_such_function” | An empty-state message appears; no unrelated result is shown. |

**UI / workflow notes:** Provide a category selector, search field, result list, and a clear empty state; selecting a result should reveal its accepted syntax.

**Validation, limitations, and open decisions:** The function catalog, category tags, case sensitivity, aliases, and whether search includes descriptions need agreement.

**Dependencies and risks:** Depends on a maintained function catalog and UI (ID 5); parser support (ID 1) must match selectable functions. Risk: search advertises unsupported syntax.

**Proposed definition of done:** Approved catalog and category map are reviewed; category, matching, and empty-state cases pass; evidence records returned functions and syntax.

## Requirement 08 – Test Evidence

**Implementation**: Function search and category filtering system allowing users to locate calculator functions by category or search term.

**Test Date**: November 8, 2026 (by Ivan Kullaya)

**Test Environment**: Web browser (Google Chrome)

| Test ID | Test Performed | Expected result | Status |
|---|---|---|---|
| R08-T1 | Select Trigonometry category and search sin | Matching trigonometric functions appear; unrelated functions are excluded | Pass |
| R08-T2 | Search for no_such_function | No matching functions message appears | Pass |
| R08-T3 | Select a function category without entering a search term | Only functions belonging to the selected category are displayed | Pass |
| R08-T4 | Select a function from the filtered results | Selected function displays its accepted syntax or inserts it into the expression field | Pass |

**Evidence:** GitHub R08 Evidence Folder

**Implementation Details:**

The function search and category filter system is designed to organize available calculator functions into categories such as trigonometry, algebra, statistics, and finance, depending on the team's approved function catalog.

Users can select a category to narrow the displayed functions and enter keywords to search for specific functions. The search system should display only functions matching the selected category and search term.

When no matching functions exist, the interface should display a clear message rather than unrelated results. Selecting a function should provide its accepted syntax to help users enter valid expressions.

**Result:** All four tests passed, confirming that Requirement 08 meets its acceptance criteria.